2️⃣

# Buổi 2: Database + Repository Pattern + Dapper

## Giáo án Buổi 2: Database + Repository Pattern + Dapper

### 0. Mục tiêu buổi học

Sau Buổi 1, bạn đã có `ProductService` dùng:

```
Minimal API → IMediator → Command/Query → Handler → ProductStore in-memory
```

Buổi 2 sẽ nâng cấp từ lưu dữ liệu tạm trong RAM sang lưu dữ liệu thật trong database.

Sau buổi này, bạn cần làm được 6 việc:

1. Hiểu vì sao `ProductStore` in-memory không phù hợp cho project thật.
2. Hiểu Repository Pattern là gì và nó đứng ở đâu trong flow CQRS.
3. Biết dùng Dapper để query database bằng SQL rõ ràng.
4. Thay `ProductStore` bằng `IProductRepository`.
5. Handler không còn phụ thuộc trực tiếp vào `ProductStore` nữa.
6. CRUD Product chạy được với database thật.

> Kết quả cuối buổi: `ProductService` lưu sản phẩm vào database thật thông qua Repository + Dapper.

---

### 1. Kiến thức nền cần hiểu trước

## 1.1. Vấn đề của ProductStore in-memory

Ở Buổi 1, ta dùng:

```
public class ProductStore
{
    public List<Product> Products { get; } = new();
}
```

Cách này giúp học flow nhanh, nhưng có nhiều vấn đề:

1. Data mất khi restart app.
2. Không phù hợp khi nhiều instance service cùng chạy.
3. Không có transaction.
4. Không query tốt khi dữ liệu lớn.
5. Handler bị phụ thuộc trực tiếp vào cách lưu dữ liệu hiện tại.

Ví dụ:

```
App restart → List<Product> rỗng lại → mất toàn bộ sản phẩm
```

Trong project thật, dữ liệu cần được lưu vào database.

---

## 1.2. Database trong Microservices

Trong microservices, một nguyên tắc phổ biến là:

```
Database per Service
```

Nghĩa là mỗi service nên sở hữu database/schema riêng của nó.

Ví dụ:

```
Catalog Service  → Catalog DB
Basket Service   → Redis / Basket DB
```

```
Order Service    → Order DB
Payment Service  → Payment DB
```

Mục tiêu:

- giảm coupling giữa services
- service tự chủ dữ liệu của nó
- tránh nhiều service cùng sửa một database chung

> Ghi nhớ: service khác không nên truy cập trực tiếp database của `ProductService`. Nếu cần dữ liệu product, nó nên gọi API/gRPC/event từ `ProductService`.

---

## 1.3. Repository Pattern là gì?

Repository Pattern là cách tạo một lớp trung gian giữa business logic và database.

Thay vì Handler gọi database trực tiếp:

```
Handler → SQL Connection → Database
```

Ta làm:

```
Handler → IProductRepository → ProductRepository → Database
```

Lợi ích:

1. Handler không cần biết SQL chi tiết.
2. Dễ đổi cách lưu dữ liệu sau này.
3. Dễ test hơn vì có thể mock `IProductRepository`.
4. Code rõ trách nhiệm hơn.

---

## 1.4. Dapper là gì?

Dapper là một micro-ORM cho .NET.

Hiểu đơn giản:

- Bạn vẫn viết SQL thật.
- Dapper giúp map kết quả SQL sang object C#.
- Ít magic hơn EF Core.
- Rất hợp khi muốn học SQL rõ ràng.

Ví dụ không dùng Dapper:

```
var reader = await command.ExecuteReaderAsync();
while (await reader.ReadAsync())
{
    var product = new Product
    {
        Id = reader.GetString(0),
        Name = reader.GetString(1)
    };
}
```

Dùng Dapper:

```
var products = await connection.QueryAsync<Product>("SELECT *
FROM Products");
```

---

## 1.5. Flow sau khi nâng cấp

Buổi 1:

```
API → MediatR → Handler → ProductStore
```

Buổi 2:

```
API → MediatR → Handler → IProductRepository → ProductRepository → Database
```

Điểm quan trọng:

- API không biết database.
- Handler không biết SQL chi tiết.
- Repository chịu trách nhiệm làm việc với DB.

---

## 2. Chọn database cho buổi học

Để học nhanh và dễ chạy, buổi này dùng SQLite.

Lý do:

- không cần cài SQL Server/PostgreSQL ngay
- chỉ tạo một file `.db`
- dễ debug
- phù hợp giai đoạn học Repository + Dapper

Sau này khi học Docker/Microservices thật, ta sẽ chuyển sang PostgreSQL hoặc SQL Server.

---

## 3. Cài package cần thiết

Trong project `ProductService`, chạy:

```
dotnet add package Dapper
dotnet add package Microsoft.Data.Sqlite
```

Kiểm tra file `.csproj` phải có package tương ứng.

---

## 4. Cập nhật appsettings.json

Mở `appsettings.json`, thêm connection string:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=products.db"
  },
  "Logging": {
```

```
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Giải thích:

```
Data Source=products.db
```

Nghĩa là SQLite sẽ tạo file database tên `products.db` ngay trong thư mục chạy app.

---

## 5. Tạo model Product chuẩn hơn

Nếu hiện tại `Product` đang nằm trong `Models/Product.cs`, giữ như sau:

```
namespace ProductService.Models;

public class Product
{
    public string Id { get; set; } = Guid.NewGuid().ToString
();
    public string Name { get; set; } = string.Empty;
}
```

Buổi này tạm giữ `Id` là `string` để ít sửa code Buổi 1.

Sau này có thể đổi sang `Guid` nếu muốn chuẩn hơn.

---

## 6. Tạo database initializer

Ta cần tạo bảng `Products` khi app start.

Tạo folder:

```
Data/
```

Tạo file `Data/DatabaseInitializer.cs`:

```
using Microsoft.Data.Sqlite;

namespace ProductService.Data;

public class DatabaseInitializer
{
    private readonly string _connectionString;

    public DatabaseInitializer(IConfiguration configuration)
    {
        _connectionString = configuration.GetConnectionString
("DefaultConnection")
            ?? throw new InvalidOperationException("Connectio
n string 'DefaultConnection' not found.");
    }

    public async Task InitializeAsync()
    {
        await using var connection = new SqliteConnection(_co
nnectionString);
        await connection.OpenAsync();

        var command = connection.CreateCommand();
        command.CommandText = """
            CREATE TABLE IF NOT EXISTS Products (
                Id TEXT PRIMARY KEY,
                Name TEXT NOT NULL
            );
            """;

        await command.ExecuteNonQueryAsync();
```

```
    }
}
```

Giải thích:

- `CREATE TABLE IF NOT EXISTS`: chỉ tạo bảng nếu chưa có.
- `Id TEXT PRIMARY KEY`: Id là khóa chính.
- `Name TEXT NOT NULL`: tên sản phẩm không được null.

---

## 7. Tạo Repository Interface

Tạo folder:

```
Repositories/
```

Tạo file `Repositories/IProductRepository.cs`:

```
using ProductService.Models;

namespace ProductService.Repositories;

public interface IProductRepository
{
    Task<List<Product>> GetAllAsync();
    Task<Product?> GetByIdAsync(string id);
    Task<Product> CreateAsync(string name);
    Task<Product?> UpdateAsync(string id, string name);
    Task<bool> DeleteAsync(string id);
    Task<List<Product>> SearchAsync(string? keyword);
    Task<int> CountAsync();
}
```

Giải thích:

Đây là contract cho nơi lưu product.

Handler chỉ cần biết interface này, không cần biết database dùng SQLite, SQL Server hay PostgreSQL.

---

## 8. Tạo ProductRepository dùng Dapper

Tạo file `Repositories/ProductRepository.cs`:

```
using Dapper;
using Microsoft.Data.Sqlite;
using ProductService.Models;

namespace ProductService.Repositories;

public class ProductRepository : IProductRepository
{
    private readonly string _connectionString;

    public ProductRepository(IConfiguration configuration)
    {
        _connectionString = configuration.GetConnectionString
("DefaultConnection")
            ?? throw new InvalidOperationException("Connectio
n string 'DefaultConnection' not found.");
    }

    private SqliteConnection CreateConnection()
    {
        return new SqliteConnection(_connectionString);
    }

    public async Task<List<Product>> GetAllAsync()
    {
        await using var connection = CreateConnection();

        var products = await connection.QueryAsync<Product>
("""
```

```
            SELECT Id, Name
            FROM Products
            ORDER BY Name;
            """);

        return products.ToList();
    }

    public async Task<Product?> GetByIdAsync(string id)
    {
        await using var connection = CreateConnection();

        return await connection.QuerySingleOrDefaultAsync<Pro
duct>("""
            SELECT Id, Name
            FROM Products
            WHERE Id = @Id;
            """, new { Id = id });
    }

    public async Task<Product> CreateAsync(string name)
    {
        var product = new Product
        {
            Id = Guid.NewGuid().ToString(),
            Name = name
        };

        await using var connection = CreateConnection();

        await connection.ExecuteAsync("""
            INSERT INTO Products (Id, Name)
            VALUES (@Id, @Name);
            """, product);

        return product;
```

```
    }

    public async Task<Product?> UpdateAsync(string id, string
name)
    {
        await using var connection = CreateConnection();

        var affectedRows = await connection.ExecuteAsync("""
            UPDATE Products
            SET Name = @Name
            WHERE Id = @Id;
            """, new { Id = id, Name = name });

        if (affectedRows == 0)
            return null;

        return await GetByIdAsync(id);
    }

    public async Task<bool> DeleteAsync(string id)
    {
        await using var connection = CreateConnection();

        var affectedRows = await connection.ExecuteAsync("""
            DELETE FROM Products
            WHERE Id = @Id;
            """, new { Id = id });

        return affectedRows > 0;
    }

    public async Task<List<Product>> SearchAsync(string? keyw
ord)
    {
        await using var connection = CreateConnection();
```

```
        if (string.IsNullOrWhiteSpace(keyword))
            return await GetAllAsync();

        var products = await connection.QueryAsync<Product>
("""
            SELECT Id, Name
            FROM Products
            WHERE LOWER(Name) LIKE '%' || LOWER(@Keyword) ||
'%'
            ORDER BY Name;
            """, new { Keyword = keyword });

        return products.ToList();
    }

    public async Task<int> CountAsync()
    {
        await using var connection = CreateConnection();

        return await connection.ExecuteScalarAsync<int>("""
            SELECT COUNT(*)
            FROM Products;
            """);
    }
}
```

---

## 9. Đăng ký Repository và DatabaseInitializer trong Program.cs

Trong `Program.cs`, thêm using:

```
using ProductService.Data;
using ProductService.Repositories;
```

Đăng ký service:

```
builder.Services.AddSingleton<DatabaseInitializer>();
builder.Services.AddScoped<IProductRepository, ProductReposit
ory>();
```

Sau khi `var app = builder.Build();`, gọi initializer:

```
using (var scope = app.Services.CreateScope())
{
    var initializer = scope.ServiceProvider.GetRequiredServic
e<DatabaseInitializer>();
    await initializer.InitializeAsync();
}
```

Phần đầu `Program.cs` sẽ gần như sau:

```
using MediatR;
using ProductService.Commands;
using ProductService.Data;
using ProductService.Queries;
using ProductService.Repositories;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssembly(typeof(Program).Assembl
y));

builder.Services.AddSingleton<DatabaseInitializer>();
builder.Services.AddScoped<IProductRepository, ProductReposit
ory>();

var app = builder.Build();

using (var scope = app.Services.CreateScope())
{
```

```
    var initializer = scope.ServiceProvider.GetRequiredServic
e<DatabaseInitializer>();
    await initializer.InitializeAsync();
}
```

> Sau khi dùng DB thật, không cần đăng ký `ProductStore` nữa.

---

# 10. Refactor Handlers dùng IProductRepository

## 10.1. CreateProductHandler

```
using MediatR;
using ProductService.Commands;
using ProductService.Models;
using ProductService.Repositories;

namespace ProductService.Handlers;

public class CreateProductHandler : IRequestHandler<CreatePro
ductCommand, Product>
{
    private readonly IProductRepository _repository;

    public CreateProductHandler(IProductRepository repositor
y)
    {
        _repository = repository;
    }

    public async Task<Product> Handle(CreateProductCommand re
quest, CancellationToken cancellationToken)
    {
        return await _repository.CreateAsync(request.Name);
```

```
    }
}
```

## 10.2. GetProductsHandler

```
using MediatR;
using ProductService.Models;
using ProductService.Queries;
using ProductService.Repositories;

namespace ProductService.Handlers;

public class GetProductsHandler : IRequestHandler<GetProducts
Query, List<Product>>
{
    private readonly IProductRepository _repository;

    public GetProductsHandler(IProductRepository repository)
    {
        _repository = repository;
    }

    public async Task<List<Product>> Handle(GetProductsQuery
request, CancellationToken cancellationToken)
    {
        return await _repository.GetAllAsync();
    }
}
```

## 10.3. GetProductByIdHandler

```
using MediatR;
using ProductService.Models;
using ProductService.Queries;
```

```
using ProductService.Repositories;

namespace ProductService.Handlers;

public class GetProductByIdHandler : IRequestHandler<GetProducu
ctByIdQuery, Product?>
{
    private readonly IProductRepository _repository;

    public GetProductByIdHandler(IProductRepository repositor
y)
    {
        _repository = repository;
    }

    public async Task<Product?> Handle(GetProductByIdQuery re
quest, CancellationToken cancellationToken)
    {
        return await _repository.GetByIdAsync(request.Id);
    }
}
```

---

## 10.4. UpdateProductHandler

```
using MediatR;
using ProductService.Commands;
using ProductService.Models;
using ProductService.Repositories;

namespace ProductService.Handlers;

public class UpdateProductHandler : IRequestHandler<UpdatePro
ductCommand, Product?>
{
    private readonly IProductRepository _repository;
```

```
    public UpdateProductHandler(IProductRepository repositor
y)
    {
        _repository = repository;
    }

    public async Task<Product?> Handle(UpdateProductCommand r
equest, CancellationToken cancellationToken)
    {
        return await _repository.UpdateAsync(request.Id, requ
est.Name);
    }
}
```

## 10.5. DeleteProductHandler

```
using MediatR;
using ProductService.Commands;
using ProductService.Repositories;

namespace ProductService.Handlers;

public class DeleteProductHandler : IRequestHandler<DeletePro
ductCommand, bool>
{
    private readonly IProductRepository _repository;

    public DeleteProductHandler(IProductRepository repositor
y)
    {
        _repository = repository;
    }

    public async Task<bool> Handle(DeleteProductCommand reque
```

```
st, CancellationToken cancellationToken)
    {
        return await _repository.DeleteAsync(request.Id);
    }
}
```

## 10.6. SearchProductsHandler

```
using MediatR;
using ProductService.Models;
using ProductService.Queries;
using ProductService.Repositories;

namespace ProductService.Handlers;

public class SearchProductsHandler : IRequestHandler<SearchPr
oductsQuery, List<Product>>
{
    private readonly IProductRepository _repository;

    public SearchProductsHandler(IProductRepository repositor
y)
    {
        _repository = repository;
    }

    public async Task<List<Product>> Handle(SearchProductsQue
ry request, CancellationToken cancellationToken)
    {
        return await _repository.SearchAsync(request.Keywor
d);
    }
}
```

## 10.7. GetProductCountHandler

Nếu bạn đã làm bài nâng cao ở Buổi 1, tạo handler như sau:

```
using MediatR;
using ProductService.Queries;
using ProductService.Repositories;

namespace ProductService.Handlers;

public class GetProductCountHandler : IRequestHandler<GetProd
uctCountQuery, int>
{
    private readonly IProductRepository _repository;

    public GetProductCountHandler(IProductRepository reposito
ry)
    {
        _repository = repository;
    }

    public async Task<int> Handle(GetProductCountQuery reques
t, CancellationToken cancellationToken)
    {
        return await _repository.CountAsync();
    }
}
```

Query tương ứng:

```
using MediatR;

namespace ProductService.Queries;

public record GetProductCountQuery : IRequest<int>;
```

Endpoint:

```
app.MapGet("/products/count", async (IMediator mediator) =>
{
    var count = await mediator.Send(new GetProductCountQuery
());
    return Results.Ok(new { count });
});
```

---

## 11. Endpoint trong Program.cs giữ nguyên gần như không đổi

Điểm hay của cách này:

```
Program.cs vẫn gọi mediator.Send(...)
Handler vẫn xử lý use case
Nhưng data source đã đổi từ ProductStore sang Database
```

Ví dụ endpoint `POST /products` vẫn như cũ:

```
app.MapPost("/products", async (CreateProductRequest request,
IMediator mediator) =>
{
    if (string.IsNullOrWhiteSpace(request.Name))
        return Results.BadRequest("Product name is require
d.");

    var product = await mediator.Send(new CreateProductComman
d(request.Name));

    return Results.Created($"/products/{product.Id}", produc
t);
});
```

Đây chính là lợi ích của tách layer tốt.

# 12. Test API sau khi dùng DB

Chạy app:

```
dotnet run
```

Tạo product:

```
POST http://localhost:5000/products
Content-Type: application/json

{
  "name": "iPhone 15"
}
```

Lấy danh sách:

```
GET http://localhost:5000/products
```

Restart app rồi gọi lại:

```
GET http://localhost:5000/products
```

Nếu dữ liệu vẫn còn, nghĩa là đã lưu DB thành công.

# 13. Bài tập chính

## 13.1. Bài 1: Thay ProductStore bằng SQLite + Dapper

Yêu cầu:

- Cài `Dapper`
- Cài `Microsoft.Data.Sqlite`
- Tạo `DatabaseInitializer`
- Tạo bảng `Products`

- Tạo `IProductRepository`
- Tạo `ProductRepository`
- Refactor toàn bộ Handler dùng `IProductRepository`
- Xóa hoặc không dùng `ProductStore` nữa

Điều kiện đạt:

```
[ ] App chạy không lỗi
[ ] Tạo product lưu vào DB
[ ] Restart app, product vẫn còn
[ ] Get/Search/Update/Delete vẫn chạy
[ ] Handler không phụ thuộc ProductStore nữa
```

---

## 13.2. Bài 2: Thêm field Price

Cập nhật `Product`:

```
public decimal Price { get; set; }
```

Cập nhật database:

```
ALTER TABLE Products ADD COLUMN Price REAL NOT NULL DEFAULT
0;
```

Vì SQLite không có migration trong bài này, cách đơn giản nhất khi học là:

1. Xóa file `products.db`
2. Sửa câu `CREATE TABLE`
3. Chạy lại app

Bảng mới:

```
CREATE TABLE IF NOT EXISTS Products (
    Id TEXT PRIMARY KEY,
    Name TEXT NOT NULL,
```

```
    Price REAL NOT NULL
);
```

Yêu cầu:

- `CreateProductRequest` có `Name`, `Price`
- `UpdateProductRequest` có `Name`, `Price`
- Repository insert/update/select đủ `Price`
- Validate `Price >= 0`

---

## 13.3. Bài 3: Thêm API lấy sản phẩm theo khoảng giá

Thêm API:

```
GET /products/by-price?min=100&max=1000
```

Yêu cầu:

- Tạo `GetProductsByPriceQuery`
- Tạo `GetProductsByPriceHandler`
- Thêm method vào `IProductRepository`

```
Task<List<Product>> GetByPriceRangeAsync(decimal min, decimal
max);
```

SQL gợi ý:

```
SELECT Id, Name, Price
FROM Products
WHERE Price >= @Min AND Price <= @Max
ORDER BY Price;
```

---

## 13.4. Bài 4: Giải thích flow mới

Trả lời bằng lời của bạn:

1. `ProductStore` có vấn đề gì?
2. `IProductRepository` giúp giảm coupling như thế nào?
3. Handler bây giờ phụ thuộc vào interface hay class cụ thể?
4. Dapper khác EF Core ở đâu?
5. Nếu sau này đổi SQLite sang PostgreSQL, ta sẽ sửa chủ yếu ở đâu?
6. Vì sao endpoint gần như không cần đổi khi đổi data source?

---

# 14. Đáp án gợi ý cho bài Price

## 14.1. Product.cs

```
namespace ProductService.Models;

public class Product
{
    public string Id { get; set; } = Guid.NewGuid().ToString
();
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
}
```

---

## 14.2. Request DTOs

```
public record CreateProductRequest(string Name, decimal Pric
e);
public record UpdateProductRequest(string Name, decimal Pric
e);
```

---

## 14.3. CreateProductCommand

```
using MediatR;
using ProductService.Models;

namespace ProductService.Commands;

public record CreateProductCommand(string Name, decimal Pric
e) : IRequest<Product>;
```

---

## 14.4. UpdateProductCommand

```
using MediatR;
using ProductService.Models;

namespace ProductService.Commands;

public record UpdateProductCommand(string Id, string Name, de
cimal Price) : IRequest<Product?>;
```

---

## 14.5. Repository CreateAsync

```
public async Task<Product> CreateAsync(string name, decimal p
rice)
{
    var product = new Product
    {
        Id = Guid.NewGuid().ToString(),
        Name = name,
        Price = price
    };

    await using var connection = CreateConnection();

    await connection.ExecuteAsync("""
```

```
        INSERT INTO Products (Id, Name, Price)
        VALUES (@Id, @Name, @Price);
        """, product);

    return product;
}
```

> Nếu làm bài này, nhớ cập nhật interface `IProductRepository` cho khớp.

---

# 15. Checklist tự chấm

Bạn đạt Buổi 2 nếu tick được:

```
[ ] Hiểu nhược điểm của ProductStore in-memory
[ ] Tạo được SQLite database file
[ ] Tạo được bảng Products
[ ] Cài và dùng được Dapper
[ ] Tạo được IProductRepository
[ ] Tạo được ProductRepository
[ ] Handler dùng IProductRepository thay vì ProductStore
[ ] CRUD vẫn chạy sau khi chuyển sang DB
[ ] Restart app dữ liệu vẫn còn
[ ] Giải thích được Repository giúp giảm coupling như thế´nào
```

---

# 16. Lỗi hay gặp

## 16.1. Không tìm thấy connection string

Lỗi:

```
Connection string 'DefaultConnection' not found.
```

Cách sửa:

- Kiểm tra `appsettings.json`
- Đảm bảo có:

```
"ConnectionStrings": {
  "DefaultConnection": "Data Source=products.db"
}
```

---

## 16.2. No such table: Products

Nguyên nhân:

- Chưa gọi `DatabaseInitializer.InitializeAsync()` khi app start.
- File DB tạo ở thư mục khác.

Cách sửa:

- Kiểm tra Program.cs đã gọi initializer chưa.
- Xóa file `.db` và chạy lại nếu schema sai.

---

## 16.3. Handler không resolve được IProductRepository

Lỗi kiểu:

```
Unable to resolve service for type 'ProductService.Repositories.IProductRepository'
```

Cách sửa:

```
builder.Services.AddScoped<IProductRepository, ProductReposit
ory>();
```

---

## 16.4. Dữ liệu mất sau restart

Nếu dữ liệu vẫn mất, có thể bạn vẫn đang dùng `ProductStore` trong Handler.

Cách kiểm tra:

- Search toàn project từ khóa `ProductStore`
- Handler không nên còn inject `ProductStore`

---

## 17. Từ vựng cần nhớ

| Từ | Nghĩa dễ hiểu |
|---|---|
| Database | Nơi lưu dữ liệu lâu dài |
| Persistence | Khả năng lưu dữ liệu bền vững |
| Repository | Lớp trung gian giữa business logic và database |
| Interface | Hợp đồng định nghĩa method cần có |
| Implementation | Class triển khai interface |
| Dapper | Micro-ORM giúp query SQL và map object |
| SQLite | Database nhẹ, lưu thành file |
| Connection String | Chuỗi cấu hình để kết nối database |
| SQL | Ngôn ngữ truy vấn database |
| Query Parameter | Tham số truyền vào SQL an toàn hơn nối chuỗi |
| Dependency Injection | Cơ chế inject dependency vào class |
| Scoped | Vòng đời service theo từng request |

---

## 18. Câu hỏi ôn tập

1. Vì sao không nên để Handler phụ thuộc trực tiếp vào `ProductStore`?
2. Repository Pattern giải quyết vấn đề gì?
3. Interface giúp giảm coupling như thế nào?
4. Dapper có nhiệm vụ gì?
5. Vì sao không nên nối chuỗi SQL trực tiếp từ input người dùng?
6. `ExecuteAsync` khác `QueryAsync` ở đâu?
7. `QuerySingleOrDefaultAsync` dùng khi nào?
8. `ExecuteScalarAsync<int>` dùng khi nào?
9. Vì sao service nên sở hữu database riêng?
10. Nếu đổi DB, phần nào của code nên thay đổi nhiều nhất?

---

# 19. Format gửi bài để được review

Khi làm xong, gửi theo mẫu:

```
Buổi 2 - Bài nộp

1. Câú trúc thư mục:
(paste ảnh hoặc text)

2. Code chính:
- Program.cs
- appsettings.json
- DatabaseInitializer.cs
- IProductRepository.cs
- ProductRepository.cs
- 1-2 Handler đã refactor

3. Test kêt quả:
- Tạo product OK chưa?
- Restart app, data còn không?
- Search/update/delete còn chạy không?

4. Cách hiêủ:
- Repository là ...
- Dapper là ...
- Flow mới là ...

5. Chô˜chưa hiêủ:
- ...
```

---

# 20. Điều kiện mở khóa Buổi 3

Bạn sẽ sang Buổi 3 khi làm được:

- CRUD Product chạy với database thật
- Handler dùng Repository thay vì ProductStore
- Giải thích được flow mới

Buổi 3 sẽ học:

```
Tách service thật: Catalog Service + Basket Service + Redis
```

Mục tiêu Buổi 3:

- Tạo service thứ hai
- Hiểu service boundary
- Basket dùng Redis
- Bắt đầu tư duy nhiều service chạy cùng lúc

# Câu hỏi & Câu trả lời ôn tập Buổi 2

## 1. Vì sao không nên để Handler phụ thuộc trực tiếp vào `ProductStore`?

Không nên để Handler phụ thuộc trực tiếp vào `ProductStore` vì `ProductStore` chỉ là nơi lưu dữ liệu in-memory.

Nó có các vấn đề:

- Dữ liệu bị mất khi app restart.
- Không phù hợp khi chạy nhiều instance.
- Không có transaction.
- Không query tốt khi dữ liệu lớn.
- Handler bị phụ thuộc trực tiếp vào cách lưu dữ liệu cụ thể.

Nếu sau này đổi từ `ProductStore` sang database thật, Handler sẽ phải sửa nhiều. Vì vậy, Handler nên phụ thuộc vào abstraction như `IProductRepository`.

---

## 2. Repository Pattern giải quyết vấn đề gì?

Repository Pattern tạo một lớp trung gian giữa Handler và database.

Thay vì Handler gọi trực tiếp database:

```
Handler → SQL / Database
```

Ta dùng:

```
Handler → IProductRepository → ProductRepository → Database
```

Repository giúp:

- Handler không cần biết SQL chi tiết.
- Handler không cần biết đang dùng SQLite, PostgreSQL hay SQL Server.
- Dễ thay đổi cách lưu dữ liệu.
- Dễ test hơn vì có thể mock interface.
- Code rõ trách nhiệm hơn.

Repository **không nên chứa business logic chính**, mà chủ yếu xử lý phần truy cập dữ liệu.

---

## 3. Interface giúp giảm coupling như thế nào?

Interface giúp các thành phần phụ thuộc vào **contract** thay vì phụ thuộc trực tiếp vào class cụ thể.

Ví dụ Handler chỉ biết:

```
IProductRepository
```

chứ không cần biết implementation bên dưới là:

```
ProductRepository
```

Nhờ vậy, nếu sau này đổi cách lưu dữ liệu từ SQLite sang PostgreSQL, ta chủ yếu sửa `ProductRepository`, còn Handler gần như không cần sửa.

Đây là cách giúp giảm coupling:

```
Handler → IProductRepository → ProductRepository
```

thay vì:

```
Handler → ProductRepository
```

---

## 4. Dapper có nhiệm vụ gì?

Dapper là một **micro-ORM** cho .NET.

Nhiệm vụ của Dapper:

- Chạy raw SQL.
- Map kết quả SQL sang object C#.
- Giúp code truy cập DB ngắn gọn hơn ADO.NET thuần.
- Ít abstraction hơn EF Core.
- Kiểm soát SQL rõ ràng hơn.

Ví dụ:

```
varproducts=awaitconnection.QueryAsync<Product>(
"SELECT Id, Name FROM Products"
);
```

Dapper không tự quản lý nhiều thứ như EF Core, nên nhanh và nhẹ hơn, nhưng dev phải tự viết SQL nhiều hơn.

---

## 5. Vì sao không nên nối chuỗi SQL trực tiếp từ input người dùng?

Không nên nối chuỗi SQL trực tiếp từ input người dùng vì dễ bị **SQL Injection**.

Ví dụ nguy hiểm:

```
varsql="SELECT * FROM Products WHERE Name = '"+keyword+"'";
```

Nếu người dùng nhập dữ liệu độc hại, họ có thể làm thay đổi câu SQL để đọc, sửa hoặc xóa dữ liệu.

Cách đúng là dùng parameter:

```
varproducts=awaitconnection.QueryAsync<Product>(
"SELECT * FROM Products WHERE Name = @Name",
new {Name=keyword }
);
```

Parameter giúp database hiểu input chỉ là dữ liệu, không phải một phần của câu lệnh SQL.

---

## 6. `ExecuteAsync` khác `QueryAsync` ở đâu?

`ExecuteAsync` dùng cho các câu lệnh không cần trả về danh sách dữ liệu, thường là:

```
INSERT
UPDATE
DELETE
```

Ví dụ:

```
awaitconnection.ExecuteAsync(
"DELETE FROM Products WHERE Id = @Id",
new {Id=id }
);
```

`QueryAsync` dùng cho câu lệnh đọc dữ liệu, thường là:

```
SELECT
```

Ví dụ:

```
varproducts=awaitconnection.QueryAsync<Product>(
"SELECT Id, Name FROM Products"
);
```

Tóm lại:

```
ExecuteAsync → ghi/sửả/xóa, không lấý list data
QueryAsync   → đọc nhiêù bản ghi từ DB
```

---

## 7. `QuerySingleOrDefaultAsync` dùng khi nào?

`QuerySingleOrDefaultAsync` dùng khi ta kỳ vọng query trả về **tối đa một bản ghi**.

Ví dụ:

```
varproduct=awaitconnection.QuerySingleOrDefaultAsync<Product>(
"SELECT Id, Name FROM Products WHERE Id = @Id",
new {Id=id }
);
```

Ý nghĩa:

- Nếu không có bản ghi nào → trả `null`.
- Nếu có đúng một bản ghi → trả object.
- Nếu có nhiều hơn một bản ghi → lỗi.

Dùng tốt cho trường hợp tìm theo `Id`, vì `Id` thường là khóa chính và chỉ có tối đa một bản ghi.

---

## 8. `ExecuteScalarAsync<int>` dùng khi nào?

`ExecuteScalarAsync<int>` dùng khi query chỉ trả về **một giá trị đơn** kiểu `int`.

Ví dụ phổ biến nhất là đếm số bản ghi:

```
varcount=awaitconnection.ExecuteScalarAsync<int>(
"SELECT COUNT(*) FROM Products"
);
```

Ngoài `COUNT(*)`, nó cũng có thể dùng cho các trường hợp như:

```
SUM(...)
MAX(...)
```

```
MIN(...)
```

miễn là query chỉ trả về một giá trị duy nhất.

---

## 9. Vì sao service nên sở hữu database riêng?

Trong microservices, mỗi service nên sở hữu database/schema riêng để giảm coupling ở mức dữ liệu.

Nếu nhiều service dùng chung một database:

- Đổi schema có thể làm hỏng service khác.
- Các service phụ thuộc chặt vào cùng một cấu trúc bảng.
- Khó deploy độc lập.
- Khó scale độc lập.
- Trách nhiệm dữ liệu bị lẫn lộn.

Ví dụ đúng hơn:

```
CatalogService → Catalog DB
BasketService  → Basket DB / Redis
OrderService   → Order DB
```

Mỗi service tự quản lý dữ liệu của nó. Service khác muốn lấy dữ liệu thì nên gọi qua API/gRPC/event, không nên đọc thẳng database của service khác.

---

## 10. Nếu đổi DB, phần nào của code nên thay đổi nhiều nhất?

Nếu đổi DB, phần thay đổi nhiều nhất nên nằm ở tầng truy cập dữ liệu.

Cụ thể:

- Connection string.
- Package/provider kết nối DB.
- `DatabaseInitializer`.

- `ProductRepository`.
- Các câu SQL đặc thù theo database.

Các phần như Endpoint, Command, Query, Handler nên ít thay đổi vì chúng đang phụ thuộc vào abstraction:

```
Endpoint → MediatR → Handler → IProductRepository
```

Còn database cụ thể nằm sau implementation:

```
IProductRepository → ProductRepository → SQLite/PostgreSQL/SQL Server
```

---

# Tổng kết flow Buổi 2

```
API Endpoint
  ↓
MediatR
  ↓
Command / Query Handler
  ↓
IProductRepository
  ↓
ProductRepository
  ↓
Database
```

Cốt lõi cần nhớ:

```
Handler không phụ thuộc trực tiếp vào DB.
Handler phụ thuộc vào interface.
Repository chịu trách nhiệm truy cập dữ liệu.
Dapper giúp chạy SQL và map dữ liệu sang object C#.
```